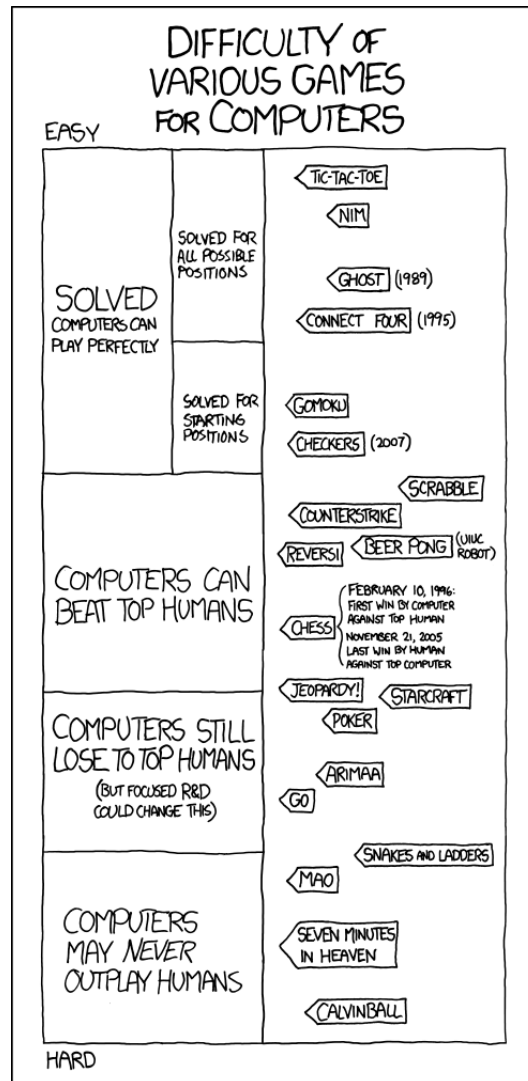


Chapitre 1

Jeux



1 Modélisation d'une famille de jeux à 2 joueurs

Un jeu est modélisé par un graphe orienté qu'on nomme **arène**. Ce graphe est caractérisé par ses sommets et ses arcs :

- Les sommets représentent les configurations de jeu (c'est à dire une description complète d'un état du jeu).
- Les arcs caractérisent la possibilité de passage d'un état du jeu à un autre : il existe un arc de u vers v si et seulement si un coup permet de passer de l'état u à l'état v .

Modèle de jeu Les jeux que nous étudierons ont quatre caractéristique principales :

- Ceux sont des jeux à deux joueurs alternants (un joueur ne peut pas jouer deux fois d'affilée) : notre graphe sera donc bipartite.
- Ceux sont des jeux à information totale c'est à dire que chaque joueur connaît la totalité des informations de la partie : les joueurs ont donc connaissance de leur emplacement dans le graphe du jeu.
- Les jeux sont sans aléa et sans mémoire.

Définition - Jeu

Un jeu est défini par un quadruplet (G, s, T_1, T_2) avec $G = (S, A)$ un graphe orienté biparti, $s \in G$ la configuration initiale du jeu et $T_1 \subset S, T_2 \subset S$ l'ensemble des états gagnants pour le joueur 1 et le joueur 2.

Définition - État final

On appelle **états finaux** les sommets puits, il y en a trois types : les éléments de T_1 , les éléments de T_2 et les autres appelés états nuls qui correspondent à un match nul.

Définition - Partie

Soit un jeu (G, s, T_1, T_2) . Une **partie** est un chemin dans G de sommet initial s et de sommet final un état terminal.

Définition - Stratégie

Soit (G, s, T_1, T_2) un jeu, avec $G = (S, A)$ tq. $S = S_1 \cup S_2$. Soit S'_1 et S'_2 les sous-ensembles de S_1 et S_2 ne contenant pas d'états puits.

On appelle **stratégie** pour le joueur i la fonction $f : S'_i \rightarrow S_{3-i}$ telle que pour tout $x \in S_i$, $(x, f(x)) \in A$.

Définition - Stratégie gagnante

Une stratégie f est dite **gagnante** pour le joueur i si quelle que soit la stratégie de l'adversaire la partie termine dans un état final de T_i .

Définition - Position gagnante

Un sommet $v \in S$ est une **position gagnante** pour le joueur i si et seulement si (G, v, T_1, T_2) admet une stratégie gagnante.

Définition - Attracteur

L'ensemble des positions gagnantes du joueur i est appelé **attracteur** de i et est noté $A(i)$.

Remarques :

- Une position gagnante du joueur i n'appartient pas forcément à S_i .
- Si un jeu n'admet pas d'état final nul, alors tous les sommets de l'arène appartiennent soit à $A(1)$, soit à $A(2)$.

2 Construction d'une stratégie optimale à l'aide du calcul des attracteurs

Proposition

Soit (G, s, T_1, T_2) un jeu. On a :

- $T_1 \subset A(1)$ et $T_2 \subset A(2)$.
- Soit $s \in S_i$. S'il existe $v \in A(i)$ tel que $(s, v) \in A$ alors $s \in A(i)$.
- Soit $s \in S_{3-i}$. Si pour tout v tel que $(s, v) \in A$, $v \in A(i)$ alors $s \in A(i)$.

Définition - Suite des attracteurs

Soit $n \in \mathbb{N}$, on définit la suite des attracteurs de i par $A_0(i) = T_i$ et

$$A_{n+1}(i) = A_n(i) \cup \{s \in S_i \mid \exists v \in A_n(i), (s, v) \in A\} \\ \cup \{s \in S'_{3-i} \mid \forall v \in S_i, (s, v) \in A \Rightarrow v \in A_n(i)\}$$

⚠ C'est une définition importante, retenez-la!

Théorème

Sur un graphe fini, la suite $(A_n(i))_{n \in \mathbb{N}}$ est stationnaire de limite $A(i)$.

DÉMONSTRATION.

On pose, pour tout $n \in \mathbb{N}$, $u_n = \text{card}(A_n(i))$. (u_n) est majorée par $\text{card}(S)$ et est croissante donc converge. De plus (u_n) est à valeur dans $\mathbb{N}$ donc est stationnaire.

La suite $(A_n(i))$ est croissante dans le sens de l'inclusion donc l'égalité des cardinaux à partir d'un certain rang la rend stationnaire.

Notons $L(i)$ sa limite. Il reste à montrer que $L(i) = A(i)$.

➤ Montrons d'abord, par récurrence, que $L(i) \subset A(i)$:

- Soit $k \in \mathbb{N}$, notons H_k : " $A_k(i) \subset A(i)$ ".
- $k = 0$: on a $A_0(i) = T_i \subset A(i)$.

H_0 est donc vraie.

- $H_k \Rightarrow H_{k+1}$: Soit $s \in A_{k+1}(i)$, on a alors 3 cas possibles :
 - Si $s \in A_k(i)$: on a le résultat par hypothèse de récurrence.
 - Si $s \in \{s \in S_i \mid \exists v \in A(i), (s, v) \in A\}$: alors il existe $v \in A(i)$ tel que $(s, v) \in A$. Ainsi le joueur i peut jouer le coup $s \rightarrow v$ pour arriver sur une position gagnante donc la position s est aussi une position gagnante : on a aussi le résultat voulu.
 - Si $s \in \{s \in S'_{3-i} \mid \forall v \in S_i, (s, v) \in A \Rightarrow v \in A_{k+1}(i)\}$ alors tous les voisins de s sont des positions gagnantes pour le joueur i , quelque soit la stratégie du joueur $3-i$, le joueur i gagne. s est donc une position gagnante.

H_{k+1} vraie.

Ce qui clôt la récurrence. On a donc bien $L(i) \subset A(i)$.

➤ Montrons ensuite, par récurrence, que $A(i) \subset L(i)$:

- Soit $j \in \mathbb{N}$, notons H_j : " $\forall s$, le joueur i gagne en au plus j coups, $s \in A_j(i)$ ".
- $j = 0$: soit s un sommet tel que le joueur i gagne en au plus 0 coup : le joueur i a déjà gagné donc $s \in T_i \subset A_0(i)$.

H_0 est vraie.

- $H_j \Rightarrow H_{j+1}$: Soit s à partir duquel on peut gagner en au plus $j+1$ coups, trois cas se présentent à nous :
 - Si on peut gagner en au plus j coups : alors par hypothèse de récurrence on peut conclure.
 - Si on a besoin des $j+1$ coups et si $s \in S_i$: alors il existe une position gagnante parmi les voisins de s qu'on note v ainsi après avoir effectué le coup (s, v) il faut j coups au joueur i pour gagner donc $v \in A_j(i)$ par hypothèse de récurrence d'où $s \in A_{j+1}(i)$ par construction de l'ensemble.
 - Si on a besoin des $j+1$ coups et si $s \in S_{3-i}$: alors quelque soit v voisin de s , v est une position gagnante pour le joueur i , de même par grâce à l'hypothèse de récurrence on obtient que $s \in A_{j+1}(i)$.

H_{j+1} vraie.

Ce qui clôt la récurrence, d'où l'inclusion réciproque.

On souhaite maintenant construire une stratégie optimale en connaissant $A(1)$ et $A(2)$. Prenons d'abord le cas d'un graphe acyclique (ce qui représente, accordons-le nous, la majorité des cas) :

Soit $s \in S$, on cherche une stratégie pour le joueur i :

- Si $s \in A(i)$: alors il existe $t \in A(i)$ tq. $(s, t) \in A$ et on pose $f(s) = t$.
- Si $s \in A(3-i)$: alors on peut définir n'importe quelle stratégie puisque le joueur i a perdu.
- Si $s \notin A(i) \cup A(3-i)$: alors il existe $t \notin A(3-i)$ tq. $(s, t) \in A$. On pose alors $f(s) = t$.

Théorème

Une telle stratégie est gagnante pour le joueur i ssi $s \in A(i)$

DÉMONSTRATION. Si on considère une partie $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_n$ où J_i suit cette stratégie, on a par rec. immédiate $\forall k \in [0; n]$, $v_k \in A(i)$. Réciproquement, on retrouve un raisonnement similaire à précédemment.

Remarque : En cas de cycle, si $s \in A(i)$ alors il existe k tel que $s \notin A_k(i)$ et $s \in A_{k+1}(1)$ et donc il existe $t \in A_k(i)$ tel que $(s, t) \in A$ et on pose $f(s) = t$ et pas n'importe quel t de $A(i)$.

3 Algorithme MinMax

Le MinMax est équivalent aux attracteurs dans un DAG (*Directed Acyclic Graph*).

Algorithme - MinMax

L'algorithme MinMax fonctionne de la manière suivante : on attribue des valeurs aux sommets. On donne à tout sommet de $A(1)$ la valeur 1, aux sommets de $A(2)$ la valeur -1 et aux autres la valeur 0. Pour savoir quelle valeur attribuer à un sommet $s \in S_1$, c'est à dire un sommet pour lequel c'est au tour du joueur 1 de faire un coup, on distingue trois cas :

- S'il existe un voisin de s dont la valeur est 1 : alors on donne au sommet la valeur 1.
- S'il existe un voisin de s avec la valeur 0 : alors on donne au sommet s la valeur 0.
- Sinon : le sommet s obtient la valeur -1 .

En définitive, le joueur 1 attribue un score à ses sommets qui est le maximum des scores de ses descendants (il maximise ses chances de gagner) et le joueur 2 lui attribue le score minimum des scores de ses voisins (il minimise ses chances de perdre).

D'où le nom de l'algorithme : MinMax.

Algorithme - MinMax_h

```

Require:  $s, p \in \mathbb{N}$ 
if  $s$  est un état final then
  return  $v(s)$ 
else if  $p = 0$  then
  return  $h(s)$ 
else if  $s \in S_1$  then
   $t \leftarrow -\infty$ 
  for each child  $n$  of  $s$  do
     $x \leftarrow \text{MinMax\_h}(n, p-1)$ 
    if  $x > t$  then
       $t \leftarrow x$ 
  return  $t$ 
else if  $s \in S_2$  then
   $t \leftarrow +\infty$ 
  for each child  $n$  of  $s$  do
     $x \leftarrow \text{MinMax\_h}(n, p-1)$ 
    if  $x < t$  then
       $t \leftarrow x$ 
  return  $t$ 

```

Ainsi, s représente le noeud/l'état actuel de notre exploration dans l'arbre, et p la profondeur. On teste bien les deux conditionnelles concernant la finalité de l'état ou la profondeur du noeud, pour ensuite savoir (dans le cas où aucune des conditionnelles n'est validée) à qui est le tour et donc savoir s'il faut maximiser le score ou justement le minimiser.

EXEMPLE. Prenons l'arbre d'appel suivant :

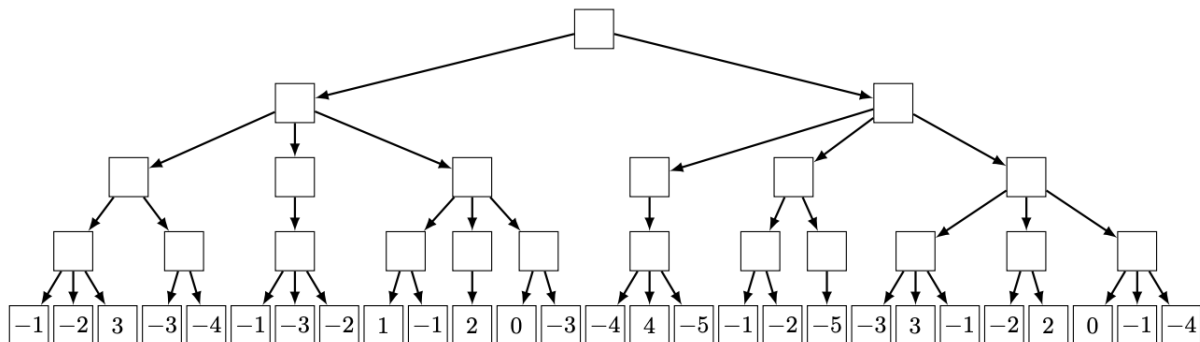


Diagram illustrating a minimax search tree structure. The tree is divided into levels by dashed lines, labeled MAX and MIN on the left, and J₁ and J₂ on the right. The root node is a MAX node (J₁). It branches into two MIN nodes (J₂). Each MIN node branches into three MAX nodes (J₁). Each MAX node branches into three MIN nodes (J₂). The leaf nodes (MIN nodes) contain numerical values representing the game state. The values are: [-1, -2, 3], [-3, -4, -1], [-3, -2, 1], [-1, 2, 0], [-3, -4, 4], [-5, -1, -2], [-5, -3, 3], [-1, -2, 2], [0, -1, -4].

[illegible]

Diagram illustrating a minimax search tree for a game. The tree structure shows alternating levels of MAX and MIN nodes, with leaf nodes representing terminal states. The values at the leaf nodes are: [-1, -2, 3, -3, -4, -1, -3, -2, 1, -1, 2, 0, -3, -4, 4, -5, -1, -2, -5, -3, 3, -1, -2, 2, 0, -1, -4]. The path from the root to the leaf node with value 2 is highlighted in red, indicating the optimal path. The values at the nodes along this path are: root (MAX), left child (MIN, value -3), left child of that (MAX, value -2), and the leaf node (2).

The diagram shows a minimax search tree with levels MAX (J1) and MIN (J2). The root node is MAX level J1 with value -3. It has two children at MIN level J2: -3 and -5. The -3 node has three children at MAX level J1: -2, -3, and an empty node. The -5 node has four children at MIN level J2: -5, an empty node, an empty node, and an empty node. The leaf nodes are at MIN level J2 and contain values from -5 to 0. A red oval highlights the root node and its right child (-5). A red triangle points to the -5 node. A pink shaded area covers the subtree rooted at the third child of the -3 node.

The diagram illustrates a minimax search tree for a 3x3 tic-tac-toe game. The tree is structured as follows:

- Level 0 (MAX, J₁):** Root node is -3.
- Level 1 (MIN, J₂):** Nodes are -3 and -5.
- Level 2 (MAX, J₁):** Nodes are -2, -3, 2, -5, -2, -2.
- Level 3 (MIN, J₂):** Leaf nodes are -1, -2, 3, -3, -4, -1, -3, -2, 1, -1, 2, 0, -3, -4, 4, -5, -1, -2, -5, -3, 3, -1, -2, 2, 0, -1, -4.

The optimal path is highlighted in blue, showing a win for player 1.

MPI* Faidherbe 2023-2025

Passons maintenant au pseudo-code (on note α avec a et β avec b) :

Algorithme - MinMax_ab

```
Require:  $(s, p) \in \mathbb{N}^2, (a, b) \in \mathbb{R}^2$   
if  $s$  est un état final then  
    return  $v(s)$   
else if  $p = 0$  then  
    return  $h(s)$   
else if  $s \in S_1$  then  
    for each child  $t$  of  $s$  do  
         $x \leftarrow \text{MinMax\_ab}(t, p-1, a, b)$   
        if  $x \geq b$  then  
            return  $b$   
        else if  $x > a$  then  
             $a \leftarrow x$   
    return  $a$   
else if  $s \in S_2$  then  
    for each child  $t$  of  $s$  do  
         $x \leftarrow \text{MinMax\_ab}(t, p-1, a, b)$   
        if  $x \leq a$  then  
            return  $a$   
        else if  $x < b$  then  
             $b \leftarrow x$   
    return  $b$ 
```